



សាកលវិទ្យាល័យភូមិន្ទភ្នំពេញ

Royal University of Phnom Penh

Identity & Access Management with authentik

goauthentik.io • MITE Final Assignment

Course	Introduction to Cryptography / Information Security
Class	MITE
Cohort	19
Instructor	Chap Chanpiseth
Group	5
Members	Seang Chanviwath Bun Chantha Lov Kimheng Seng Phirum
Deployment	Docker Compose lab in this folder

Part 1: Research & Conceptual Understanding

1. What Is an Identity Provider?

An Identity Provider (IdP) is a trusted system that verifies who a user is and gives applications a reliable identity result. Instead of each application keeping separate user databases and password rules, the IdP centralizes login, account lifecycle, access policy, and session control. authentik fits this role because it is designed as an open-source IdP and Single Sign-On platform with support for OAuth2/OpenID Connect, SAML2, LDAP, RADIUS, and related identity workflows.

Centralized authentication benefits an organization in several practical ways. First, it reduces password sprawl because users authenticate through one controlled platform rather than many unrelated application logins. Second, it lets security teams apply consistent requirements such as MFA, password policy, recovery flows, and

conditional access. Third, it improves auditability: authentik records user and system events, so administrators can review who logged in, which applications were accessed, and which configuration changes occurred. Finally, it simplifies onboarding and offboarding because group membership and application access can be managed in one place.

In this lab, authentik is the IdP and Grafana is the relying application. Users authenticate to authentik, authentik validates the account and policy, and Grafana accepts the identity information through OIDC.

2. OAuth2/OIDC vs. SAML2

OAuth2 is primarily an authorization framework, while OpenID Connect adds authentication and identity claims on top of OAuth2. In an OIDC web login, the browser is redirected from the application to authentik, authentik authenticates the user, returns an authorization code, and the application exchanges that code for tokens. The common token format is JSON Web Token (JWT), and OIDC exposes discovery metadata, userinfo, token, revoke, introspection, and JWKS endpoints.

SAML2 is older and XML-based. A SAML Service Provider redirects the user to the IdP, and the IdP returns a signed XML assertion to the Service Provider. SAML commonly appears in enterprise SaaS and legacy corporate systems. It relies heavily on XML metadata, bindings such as HTTP Redirect or HTTP POST, certificates, assertion signing, and explicit trust between the IdP and Service Provider.

OIDC is often preferred for modern web apps, APIs, mobile apps, and cloud-native systems because JSON tokens and discovery endpoints are easier for developers to consume. SAML is often preferred when integrating with older enterprise software or SaaS platforms that already standardize on SAML. Both can be secure when implemented correctly. OIDC security depends on redirect URI validation, client secret protection, short token lifetimes, PKCE where appropriate, and signature validation. SAML security depends on certificate management, assertion audience validation, clock synchronization, and avoiding weak XML parsing or unsigned assertion acceptance.

3. Self-Hosting vs. Cloud IdP

Self-hosting authentik gives the organization direct control over identity data, deployment location, integrations, and policy customization. This supports data sovereignty because user identities, logs, group structures, and application metadata can remain inside an approved environment. It can also reduce licensing cost for labs, small teams, and organizations that prefer open-source infrastructure.

The operational trade-off is responsibility. With **cloud IdP**, the provider handles availability, patching, scaling, and much of the compliance evidence. With self-hosting, the organization must secure the host, manage backups, monitor logs, rotate secrets, update containers, and test restore procedures. For regulated or highly customized environments, that control can be a strength. For teams without operational capacity, it can become a risk. A production authentik deployment should therefore include TLS, reverse proxy hardening, tested backups, MFA for administrators, centralized log forwarding, and documented upgrade procedures.

Part 2: Deployment Lab

Environment Setup

The lab uses Docker Compose for repeatable deployment. The stack contains:

Service	Purpose
<code>`postgresql`</code>	Stores authentik users, policies, providers, sessions, events, and configuration
<code>`server`</code>	authentik web/API service
<code>`worker`</code>	authentik background tasks, blueprints, outpost/certificate tasks
<code>`grafana`</code>	OIDC relying party sample application
<code>`backup`</code>	Optional scheduled backup job

Run:

```
docker compose pull
docker compose up -d
docker compose ps
```

The `.env` file contains generated values for `PG_PASS`, `AUTHENTIK_SECRET_KEY`, demo user passwords, and OIDC client secret. These follow the official requirement to generate a PostgreSQL password and authentik secret key before startup.

Initial Configuration

Open:

<http://localhost:9000/if/flow/initial-setup/>

Create the `akadmin` password if the initial setup flow appears. The included blueprint also creates `demo-admin` in the `assignment-admins` superuser group for demonstration.

User & Group Management

The blueprint creates:

Object	Purpose
<code>`demo-admin`</code>	Administrator account
<code>`demo-user`</code>	Regular user with Grafana access
<code>`demo-outsider`</code>	Regular user without Grafana access, used for denial testing
<code>`assignment-admins`</code>	Superuser group
<code>`grafana-access`</code>	Group allowed to access Grafana

This covers the required administrator account, regular account, group creation, group membership, and permission assignment.

MFA

For MFA evidence, log in as `demo-admin`, enroll a TOTP authenticator from the user settings/authenticator enrollment area, scan the QR code with Google Authenticator/Authy/compatible app, and verify a fresh login requires the TOTP code.

Screenshots To Insert

Screenshot	Caption
Docker Compose status	Shows authentik, PostgreSQL, worker, and Grafana containers running
Initial setup	Shows the first setup flow or completed admin bootstrap
Users and groups	Shows `demo-admin`, `demo-user`, and `grafana-access` membership
TOTP enrollment	Shows admin MFA enrollment and successful MFA login

Part 3: Application Integration

OAuth2/OIDC Provider

The file blueprints/assignment-lab.yaml creates an OAuth2/OIDC provider named grafana-oidc-provider. It uses:

Setting	Value
Client ID	`OIDC_CLIENT_ID` from `.env`
Client Secret	`OIDC_CLIENT_SECRET` from `.env`
Redirect URI	`GRAFANA_REDIRECT_URI` from `.env`
Scopes	`openid`, `profile`, `email`
Authorization flow	`default-provider-authorization-implicit-consent`
Grant types	Authorization code and refresh token

Discovery endpoint:

<http://localhost:9000/application/o/grafana/.well-known/openid-configuration>

Sample Application

Grafana is configured in docker-compose.yml using environment variables:

Grafana setting	Purpose
`GF_AUTH_GENERIC_OAUTH_ENABLED`	Enables OAuth login
`GF_AUTH_GENERIC_OAUTH_AUTH_URL`	Browser authorization redirect to authentik
`GF_AUTH_GENERIC_OAUTH_TOKEN_URL`	Back-channel token exchange
`GF_AUTH_GENERIC_OAUTH_API_URL`	UserInfo lookup
`GF_AUTH_GENERIC_OAUTH_SCOPES`	Requests OIDC identity claims

SSO Demonstration

Open <http://localhost:3000>, choose authentik login, and authenticate as demo-user. The browser redirects to authentik, returns to Grafana through `/login/generic_oauth`, and Grafana creates a session from the OIDC claims. In authentik, the session and events can be reviewed under the Admin UI events/session views.

Conditional Access Policy

The blueprint creates an expression policy named grafana-access-group-only:

```
if request.user.is_anonymous:
    return False

return ak_is_group_member(request.user, name="grafana-access")
```

The policy is bound to the Grafana Demo application. demo-user is allowed because the account is in grafana-access. demo-outsider is denied because it is not in the group.

Screenshots To Insert

Screenshot	Caption
Provider settings	Shows OIDC client, redirect URI, and selected scopes
Grafana login redirect	Shows user being redirected to authentik
Successful SSO	Shows Grafana session after login as `demo-user`
Denied access	Shows `demo-outsider` blocked by conditional access
Session list/events	Shows authentik recording the login/session

Part 4: Backup and Restore

Task 1: What Must Be Backed Up

authentik state is split between PostgreSQL and mounted static directories. PostgreSQL is the critical component because it stores users, groups, applications, providers, policies, events, tokens, and configuration. Static directories preserve local files used by authentik and this lab:

Path	Reason
<code>/data`</code>	Uploaded icons, flow backgrounds, reports, and media under the current storage layout
<code>/certs`</code>	Filesystem certificates used for discovery/import
<code>/custom-templates`</code>	UI/template customizations
<code>/blueprints`</code>	Infrastructure-as-code configuration for the lab

Task 2: Manual PostgreSQL Backup

Run:

```
./scripts/backup-now.sh
```

This creates:

```
backups/<timestamp>/authentik.dump
```

The dump uses PostgreSQL custom format so it can be restored with `pg_restore`.

Task 3: Static Volume Backup

The same script creates:

```
backups/<timestamp>/static.tar.gz
```

This archive includes data, certs, custom-templates, and blueprints.

Task 4: Simulate Data Loss And Restore

Create a backup first, then delete the demo user:

```
./scripts/backup-now.sh
./scripts/simulate-data-loss.sh
```

Restore:

```
./scripts/restore-db.sh backups/<timestamp>/authentik.dump
```

After restoration, confirm demo-user exists again and can log in to Grafana.

Task 5: Automated Recurring Backups

Start the scheduled backup profile:

```
docker compose --profile backup up -d backup
```

Retention is controlled by .env:

```
BACKUP_INTERVAL_SECONDS=86400  
BACKUP_RETENTION_DAYS=7
```

This produces a daily backup and deletes timestamped backup folders older than the retention period.

Part 5: Security Analysis

1. Audit Logging

authentik event logging captures user and system actions. In the Admin UI, events are visible under Events > Logs and also appear in dashboard recent events. Examples include successful login, failed login, policy execution result, user or group modification, provider/application changes, token activity, and administrative configuration updates. Three specific examples for this lab are: login success for demo-user, denied policy evaluation for demo-outsider, and blueprint-created object changes for the Grafana application/provider.

Comprehensive audit logging is important for compliance because regulated environments must prove who accessed systems, when access occurred, and what administrative changes were made. It is also essential for incident response. If an account is abused, logs help determine the first suspicious event, affected applications, source IP information, and whether policies or credentials were changed.

2. GeoIP & Impossible Travel Detection

authentik supports GeoIP context on login, authorization, and enrollment events. GeoIP policies can use country, ASN, distance checks, historical login count, and impossible-travel checks. The threat model is suspicious access from geographically inconsistent locations. For example, if a user logs in from Phnom Penh and then ten minutes later logs in from a distant country, the required travel speed is impossible. A GeoIP policy can fail the request or trigger additional control, reducing the risk from stolen credentials.

3. Zero Trust Architecture

Zero Trust assumes no user, device, or network is trusted by default. authentik supports this model through centralized application access policy and MFA. The Grafana policy in this lab limits access to members of grafana-access, so having a valid password alone is not enough. MFA further limits implicit trust by requiring a second factor for administrator login. authentik also supports event logs, session review, provider-specific authorization flows, and conditional policies, which let administrators continuously evaluate access instead of granting permanent broad trust.

4. Threat Scenario Analysis

A realistic threat is credential stuffing against externally exposed applications. Attackers try leaked username/password pairs across many services. With authentik, applications delegate login to one controlled IdP, so defenses can be concentrated. MFA blocks many stolen-password attempts, logs show failed authentication patterns, and policies can restrict access by group or risk signals. If attackers successfully authenticate with a low-privilege account, application policies still limit which apps they can open. Administrators can review events, identify repeated failures or unusual source locations, disable the account, reset credentials, revoke sessions/tokens, and preserve evidence for incident response.

This layered response is stronger than isolated application logins because detection, policy enforcement, and recovery happen in one place.

Reflection

This assignment shows that identity management is not only a login screen. An IdP becomes a security control plane for users, groups, applications, protocols, sessions, logs, and recovery. The most useful lesson was seeing how OAuth2/OIDC depends on multiple moving parts: redirect URIs, client secrets, scopes, token exchange, and application-side claim handling. Another important lesson was backup design. For an identity platform, a failed restore is not a minor inconvenience; it can block access to many dependent systems.

In a production deployment, I would place authentik behind a TLS reverse proxy, use DNS names rather than localhost URLs, store secrets in a secret manager, forward logs to a SIEM, enforce MFA for all privileged accounts, use tested off-host backups, document upgrade rollback procedures, and run restore drills. I would also avoid broad default access by making each application require an explicit group or entitlement.

I would also separate development and production configuration. The lab uses simple passwords and local bind mounts for repeatability, but production should use stronger secret handling, restricted administrator roles, network segmentation, health monitoring, and alerting for high-risk events such as repeated failures, impossible travel, and privileged configuration changes. I would test user lifecycle processes too: onboarding, group changes, account disabling, and emergency administrator recovery.

References

- authentik Docker Compose installation: <https://docs.goauthentik.io/install-config/install/docker-compose/>
- authentik automated install: <https://docs.goauthentik.io/install-config/automated-install/>
- authentik OAuth2/OIDC provider: <https://docs.goauthentik.io/add-secure-apps/providers/oauth2/>
- authentik SAML provider: <https://docs.goauthentik.io/add-secure-apps/providers/saml/>
- authentik blueprints: <https://docs.goauthentik.io/customize/blueprints/>
- authentik backup and restore: <https://docs.goauthentik.io/sys-mgmt/ops/backup-restore/>
- authentik events/logging: <https://docs.goauthentik.io/sys-mgmt/events/>
- authentik GeoIP policy: <https://docs.goauthentik.io/customize/policies/types/geoip/>

Evidence Appendix

The screenshots below were captured by scripts/run-report-automation.ps1 from the local Docker Compose lab.

Screen recording: docs\videos\sso-login-and-denied-flow.webm

01-docker-compose-ps.png

Docker Compose status showing authentik, PostgreSQL, worker, Grafana, and backup services.

Docker Compose Status

Captured 2026-05-28 15:34:12

Command

```
docker compose ps
```

Output

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
authentik-backup-1	docker.io/library/postgres:16-alpine	"docker-entrypoint.sâ€¦!"	backup	27 hours ago	Up 39 minutes	5432/tcp
authentik-grafana-1	docker.io/grafana/grafana-oss:12.0.2	"/run.sh"	grafana	28 hours ago	Up 39 minutes	0.0.0.0:3000->3000/tcp,
[::]:3000->3000/tcp						
authentik-postgresql-1	docker.io/library/postgres:16-alpine	"docker-entrypoint.sâ€¦!"	postgresql	28 hours ago	Up 39 minutes (healthy)	5432/tcp
authentik-server-1	ghcr.io/goauthentik/server:2026.5.0	"dumb-init -- ak serâ€¦!"	server	28 hours ago	Up 39 minutes (healthy)	0.0.0.0:9000-
>9000/tcp, [::]:9000->9000/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp						
authentik-worker-1	ghcr.io/goauthentik/server:2026.5.0	"dumb-init -- ak worâ€¦!"	worker	28 hours ago	Up 39 minutes (healthy)	

02-initial-setup.png

Initial setup or completed bootstrap evidence for the authentik instance.

Initial Setup and Bootstrap Evidence

Captured 2026-05-28 15:34:13

Bootstrap verified

Check	Result
akadmin user exists	yes
demo-admin user exists	yes
demo-admin confirmed TOTP devices	1
Grafana application exists	yes
Grafana OIDC provider exists	yes

03-users-groups.png

Demo users, groups, and Grafana access membership used in the lab.

Users and Groups Evidence

Captured 2026-05-28 15:34:13

Live authentik data

Users

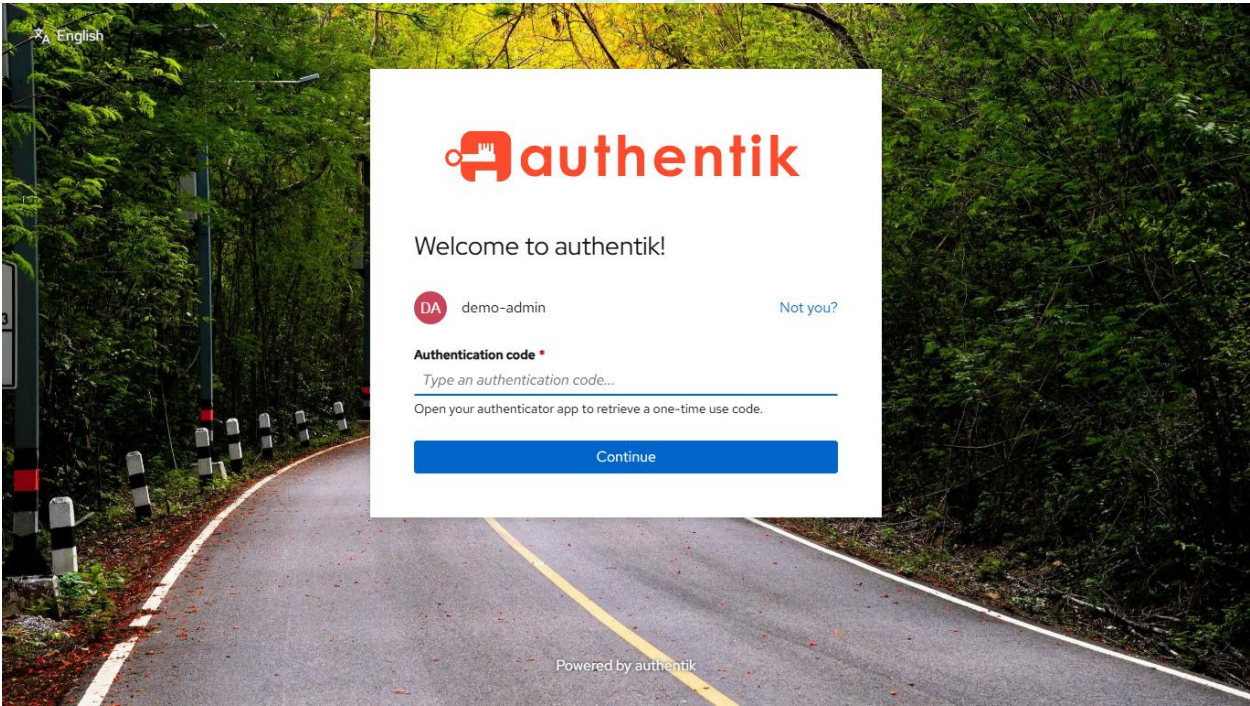
Username	Name	Email	Active	Type
demo-admin	Demo Administrator	demo-admin@example.local	yes	internal
demo-outsider	Demo Unauthorized User	demo-outsider@example.local	yes	internal
demo-user	Demo Regular User	demo-user@example.local	yes	internal

Groups

Group	Superuser	Members
assignment-admins	yes	demo-admin
grafana-access	no	demo-admin, demo-user

04-admin-totp-enrolled.png

Administrator MFA proof showing the TOTP authentication challenge for demo-admin.



05-grafana-provider.png

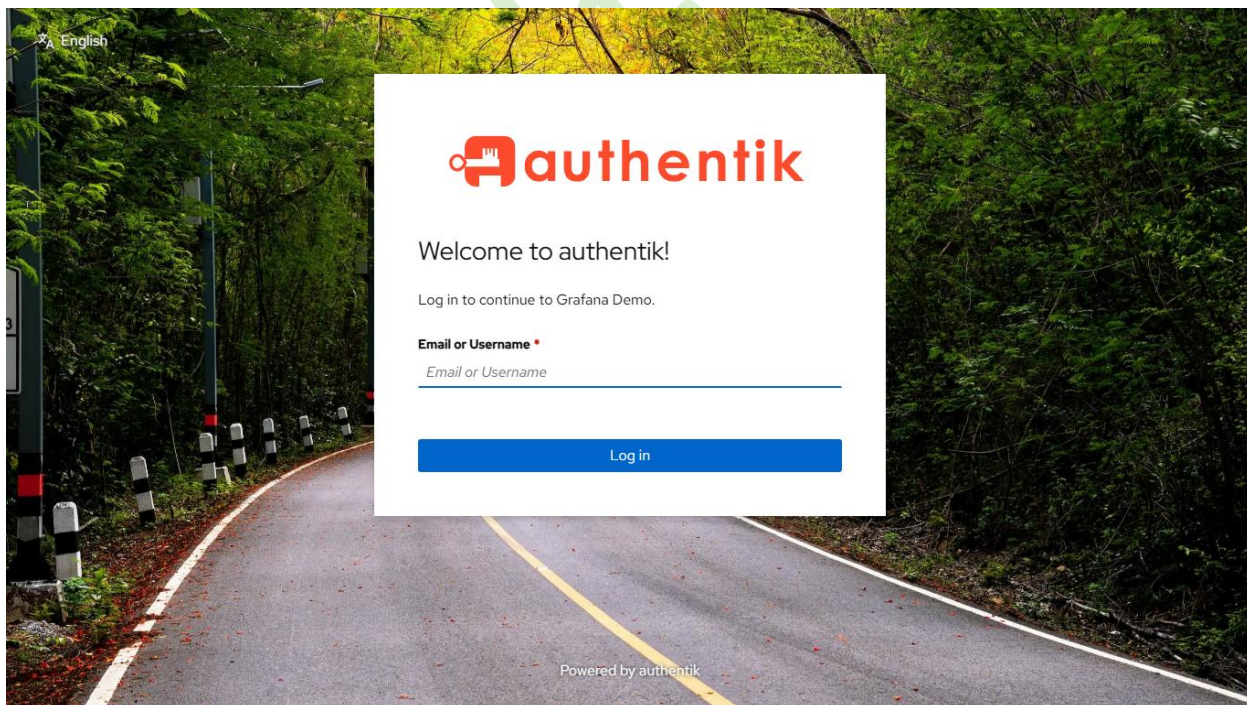
OAuth2/OIDC provider details for the Grafana integration.

The screenshot displays the Authentik web interface for configuring an OAuth2/OIDC provider. The left sidebar shows the navigation menu with 'Providers' selected. The main content area is titled 'grafana-oidc-provider' and includes tabs for 'Overview', 'Preview', 'Changelog', and 'Permissions'. The 'Overview' tab is active, showing the following configuration details:

- Info**
 - Name:** grafana-oidc-provider
 - Assigned to application:** Grafana Demo
 - Client Type:** Confidential
 - Client ID:** grafana-demo
 - Redirect URIs:** strict: http://localhost:3000/login/generic_oauth
 - Logout URI:** -
 - Logout Method:** -
- OpenID Configuration URL:** http://localhost:9000/application/o/grafana/.well-known/openid-configuration
- OpenID Configuration Issuer:** http://localhost:9000/application/o/grafana/
- Authorize URL:** http://localhost:9000/application/o/authorize/
- Token URL:** http://localhost:9000/application/o/token/
- Userinfo URL:** http://localhost:9000/application/o/userinfo/
- Logout URL:** http://localhost:9000/application/o/grafana/end-session/

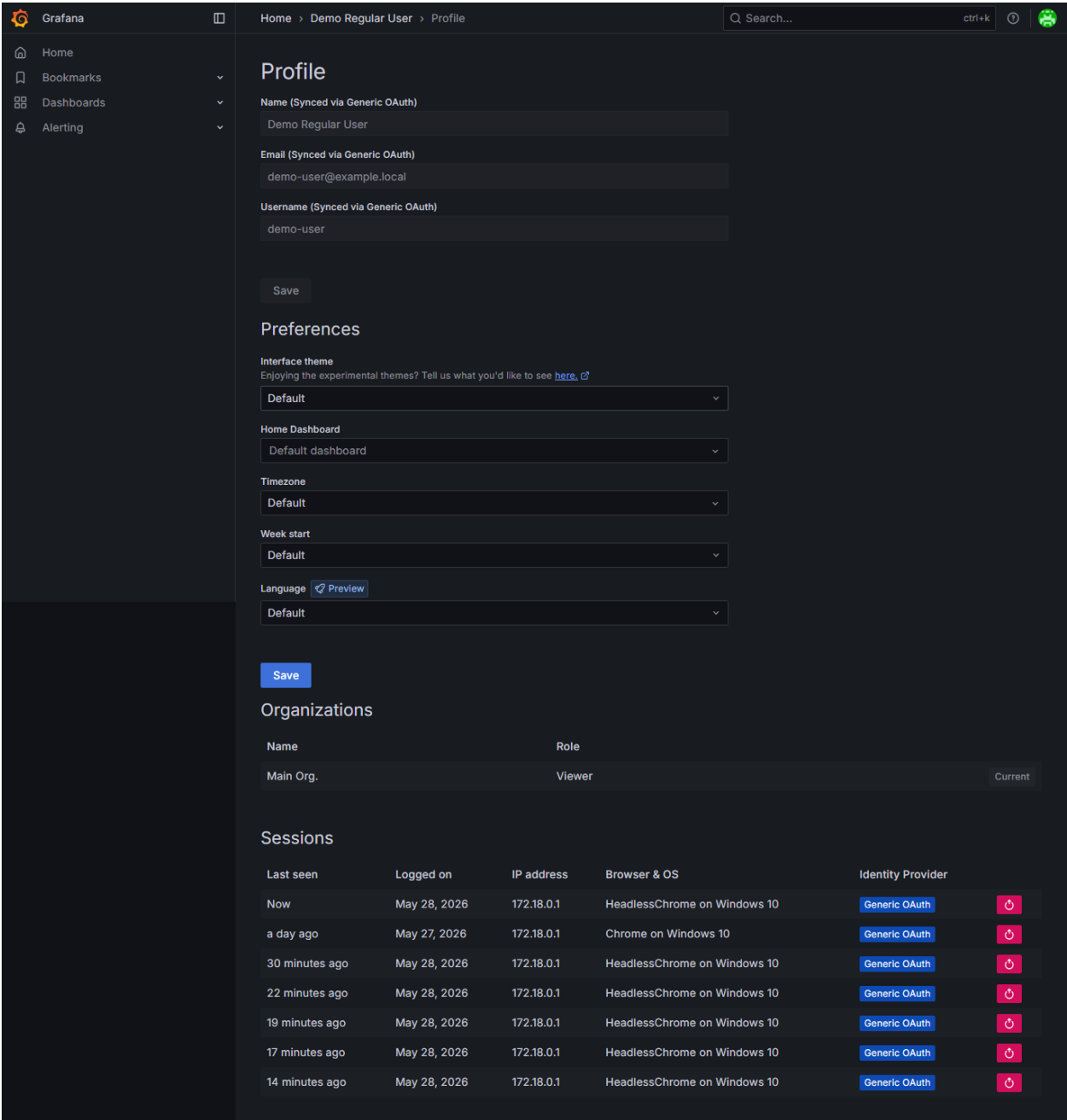
06-grafana-login-flow.png

Grafana redirecting the user to authentik for SSO.



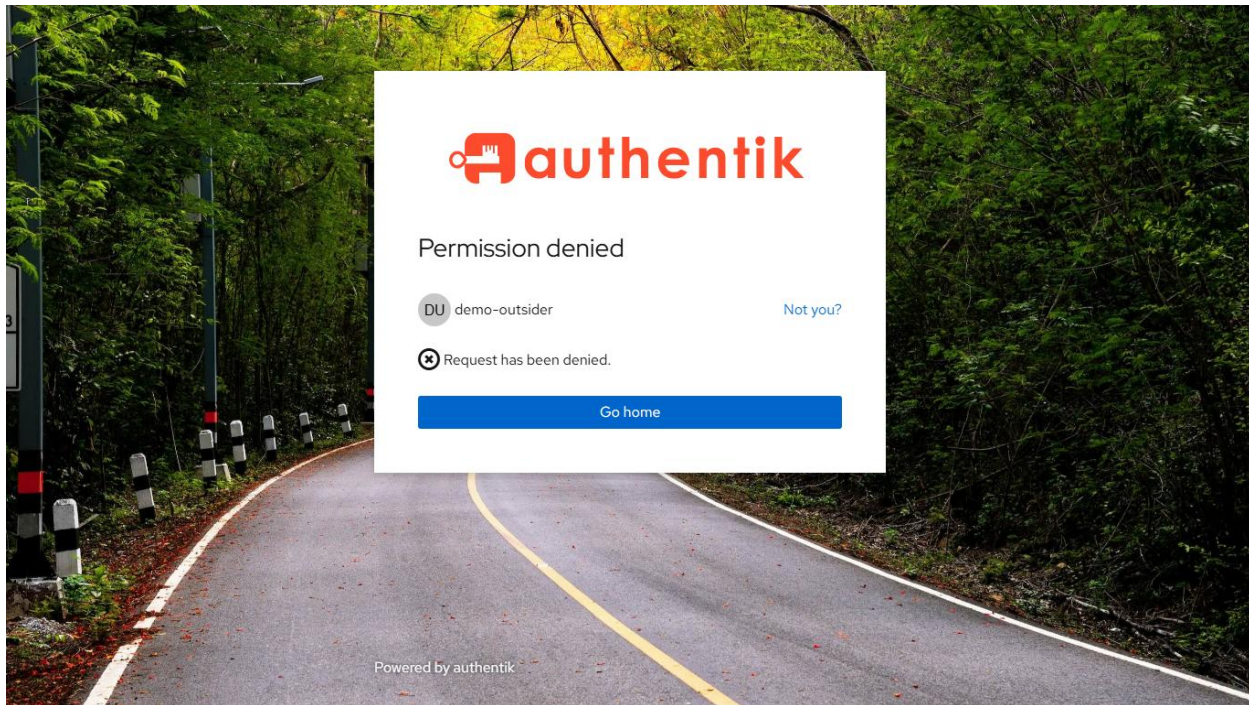
07-grafana-sso-success.png

Successful Grafana SSO session as demo-user.



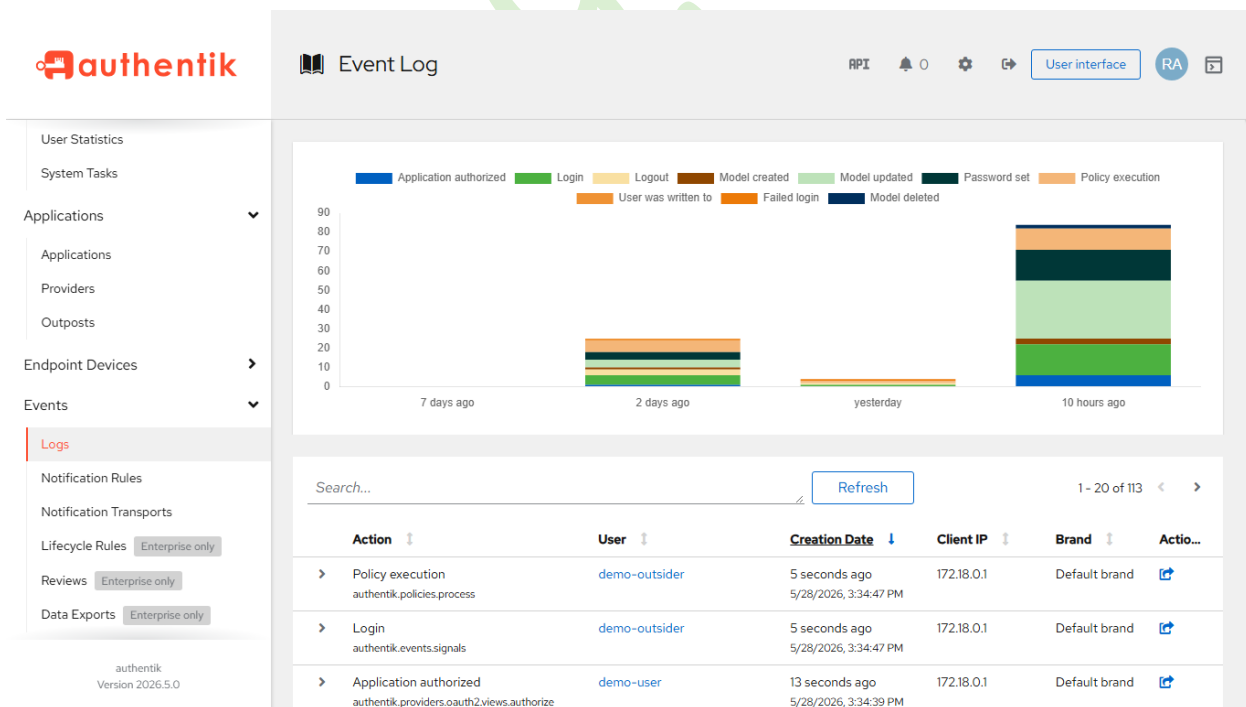
08-conditional-access-denied.png

Conditional access policy denying demo-outsider.



09-events-logs.png

authentik event log entries for login and policy activity.



10-backup-restore.png

Backup output and restore validation evidence.

Backup and Restore Validation

Captured 2026-05-28 15:34:14

Command

```
scripts/backup-now.ps1 -Stamp 20260528T083401Z; pg_restore into temporary database
```

Output

```
Backup written to backups\20260528T083401Z  
Database dump: backups\20260528T083401Z\authentik.dump  
Static archive: backups\20260528T083401Z\static.tar.gz  
  
Restore validation succeeded in temporary database: authentik_restore_check_20260528t083401z
```